Morning Brief

Ludovici · Sunday, June 14, 2026

In this edition

By Stephanie Ludovici

A personal learning and awareness brief: AI governance, cybersecurity, project status, five new project ideas, and four training items. Built to keep me current and sharp.

AI governance & policy

- **OWASP publishes *State of Agentic AI Security and Governance* v2.01 (Jun 1, 2026).** The update adds a live incidents-and-exploits tracker, a revised agent taxonomy, an enterprise adoption maturity model, and new sections on agent identity, AI SBOM, and supply-chain provenance. The headline finding: nearly every threat the 2025 paper called *plausible* now has a CVE, vendor advisory, or production incident attached. This is the exact triangulation your v3.0 agentic overlay needs — and the live tracker is a ready-made evidence base beyond OWASP's static Top 10. OWASP report
- **NIST's AI Agent Standards Initiative (via CAISI) firms up; an AI Agent Interoperability Profile is slated for Q4 2026.** The initiative targets agent identity and authorization, security and risk management, and monitoring/logging — the first authoritative U.S. counterweight to OWASP for agentic governance, and a direct answer to the "OWASP is the lone surviving source" gap in your V3.0 tracker. NIST AI RMF

- **Bipartisan *Great American AI Act* discussion draft released (Jun 4, 2026)** by Reps. Obernolte (R-CA) and Trahan (D-MA) — billed as the most comprehensive bipartisan AI regulatory framework proposed to date. Worth reading for how it treats federal use, evaluation, and standards roles (NIST/CAISI). Akin analysis
- **A June 2026 AI Executive Order ties AI to federal cyber defense.** It tasks Treasury (with the National Cyber Director, CISA, NSA) to stand up a vulnerability clearinghouse within 30 days, and Treasury/NSA/CISA/NIST to define classified benchmarks for "covered frontier models" within 60 days — a concrete fusion of AI governance and cyber operations relevant to any IC-facing framework. Wiley alert
- **EU AI Act "Digital Omnibus" moves toward formal adoption (provisional deal May 7, 2026).** High-risk obligations for biometrics, critical infrastructure, employment, and migration shift to Dec 2, 2027; new "nudifier"/CSAM prohibitions take effect Dec 2, 2026; transparency rules still land August 2026. The deadline reshuffle is the live high-risk baseline to benchmark tier definitions against. Covington / Inside Privacy

---

Cybersecurity & threats

- **LiteLLM AI-gateway flaw under active exploitation — CISA KEV add (Jun 8, 2026).** CVE-2026-42271 is a command-injection bug in LiteLLM's MCP preview endpoints (auth'd users can spawn subprocesses on the proxy host); chained with CVE-2026-48710 (a Starlette Host-header bypass) it becomes **unauthenticated RCE**. Successful exploitation exposes every connected model-provider credential and pivots into downstream AI infrastructure. Directly on your threat model: this is the prompt-injection/agent-supply-chain risk class made concrete, and it touches the MCP layer this workspace itself runs on. *Mitigation: patch to LiteLLM ≥ 1.83.7.* The Hacker News · Help Net Security
- **Check Point Security Gateway improper-auth bug added to KEV (Jun 8, 2026).** CVE-2026-50751 affects a widely deployed perimeter device — high federal-estate relevance; prioritize against the gateway fleet. CISA KEV (Jun 8)
- **Palo Alto Networks flags a high-severity flaw in Cortex XSOAR / XSIAM (CVE-2026-0274)** — improper credential

validation in the Commvault SecurityIQ integration. A vulnerability in SOC tooling is force-multiplying; worth tracking for any estate running these platforms. Digital Forensics Magazine roundup

- **CISA reverses course on its planned cybersecurity-advisory format changes (~Jun 12, 2026).** After pushback, CISA will keep advisories on established channels rather than narrowing distribution — relevant to how the federal community ingests authoritative cyber guidance. Infosecurity Magazine
- **Australia's ACSC warns CVE-2026-41940 in cPanel/WHM is under active exploitation.** Hosting-panel RCE with broad internet exposure; a useful reminder to confirm no shadow cPanel instances sit inside the boundary. Digital Forensics Magazine roundup

Project status

- **NetMon-Server** *(most active — edited today)* — Single-file PowerShell SIEM/network monitor (`netmon-server.ps1`, v0.40.3, loopback-only). The dashboard **SIEM redesign is design-complete**: hi-fi mockup approved, developer handoff package (`handoff/`) and an untouched `initial-version/` rollback snapshot are in place for Claude Code to implement. **Next action:** given this week's LiteLLM KEV, add a detection/posture rule that flags any locally reachable AI-gateway/MCP service (e.g., LiteLLM on a dev port) and surfaces unpatched-KEV exposure on the host.
- **MongoDB (vetting-pipeline discovery)** — Discovery research tracker plus illustrated process artifacts (DFDs, two-cycle and dual-path figures in SVG/Mermaid), last touched Jun 12. **Next action:** convert `vetting_research_tracker.md` open threads into a one-page data-lineage diagram for the pipeline — ties straight to today's data-centric training item on data-quality dimensions.
- **AI Framework (v2.x shipped; v3.0 tracked)** — `V3.0-ROADMAP-FOLLOWUP.md` keeps four deferred threads alive: categorical pre-screen (prototyped), the agentic overlay (OWASP single-source), GenAI groundedness, and principles/modality validation. **Next action:** log today's OWASP v2.01 report and the NIST CAISI AI Agent Standards Initiative into the agentic-overlay evidence base — both directly close the "verify

NIST/academic agent-eval guidance to triangulate beyond OWASP" task the tracker calls out.

Five project ideas

Full write-ups (problem statement, objective, method, duration, sources) saved to `project-ideas/2026-06-14-ideas.md`. Summary:

1. **Physics-Informed Kolmogorov–Arnold Networks (PIKANs) for PDEs** — Physics · Python (`pykan`), Julia optional. *Problem:* MLP-based PINNs struggle with sharp/oscillatory solutions and interpretability. *Objective:* compare PIKAN vs MLP-PINN on Burgers' & Poisson for accuracy, params, and cost. *Method:* PDE-residual + BC loss; relative-L2 vs numerical reference; accuracy-vs-FLOPs Pareto. *Duration:* 3–4 weeks.
2. **Equivariant diffusion model for 3-D molecular conformer generation** — Comp chemistry · Python (PyTorch/`e3nn`). *Problem:* cheap conformer samplers explore space crudely; big generators don't fit consumer GPUs. *Objective:* train a small E(3)-equivariant diffusion model on a QM9/GEOM subset and beat RDKit ETKDG on RMSD coverage. *Method:* equivariant denoising over coordinates; validity/uniqueness + min-RMSD metrics. *Duration:* 5–7 weeks.
3. **Last-layer Laplace approximation for calibrated land-cover classification** — Remote sensing · Python (`laplace-torch`). *Problem:* satellite-image CNNs are overconfident; full Bayesian DL is too costly. *Objective:* add post-hoc Bayesian uncertainty to an EuroSAT CNN and improve ECE + OOD detection without retraining. *Method:* last-layer Laplace (inverse-Hessian/GGN); compare vs temperature scaling on ECE/NLL/AUROC. *Duration:* 2–3 weeks. *(Today's notebook demonstrates the core method.)*
4. **INLA-SPDE spatial model for PM2.5 air-quality downscaling** — Environmental epidemiology · R (`R-INLA`). *Problem:* sparse monitors give exposure estimates with unreported uncertainty; MCMC is slow on fine grids. *Objective:* downscale PM2.5 to a continuous surface with calibrated intervals; validate coverage by spatial CV. *Method:* SPDE + INLA Matérn random field vs ordinary kriging. *Duration:* 3–4 weeks.
5. **Hamiltonian Neural Networks for conserved-quantity discovery** — Physics/dynamical systems · Julia (SciML) + Python cross-check. *Problem:* generic neural surrogates drift

because they ignore conservation laws. *Objective:* learn a scalar Hamiltonian for pendulum/two-body systems and beat a neural-ODE baseline on long-rollout error and energy drift. *Method:* parameterize H(q,p), get derivatives via autodiff, integrate Hamilton's equations. *Duration:* 3–4 weeks.

Training

1. Python — `dataclasses` for clean, correct data containers `@dataclass` (Python 3.7+) auto-generates `__init__`, `__repr__`, and `__eq__` from typed class attributes, so you describe *what a record holds* instead of writing boilerplate. It matters because hand-written value objects are where silent bugs hide: a forgotten field in `__eq__`, a `__repr__` that drifts from the attributes, an `__init__` that doesn't match. Dataclasses keep all of that in sync and make intent readable.

The classic pitfall is **mutable default arguments**. Writing `tags: list = []` shares one list across every instance — a bug dataclasses actually refuse to let you make. Use `field(default_factory=list)` instead, which builds a fresh object per instance.

```
# Demonstrate a dataclass with a safe mutable default and
from dataclasses import dataclass, field


@dataclass(frozen=True)  # frozen=True makes instances im
class Measurement:
    sensor_id: str
    value: float
    tags: list[str] = field(default_factory=list)  # Fres

    def is_valid(self) -> bool:
        # Methods work normally on a dataclass.
        return self.value >= 0.0


m1 = Measurement("s-01", 3.2, tags=["calibrated"])
m2 = Measurement("s-01", 3.2, tags=["calibrated"])
print(m1 == m2)        # True: equality is generated from
print(m1.is_valid())   # True
```

`frozen=True` is worth reaching for when a record should not change after creation — it prevents accidental mutation and makes instances usable as dict keys or set members.

Reference: Python docs — `dataclasses`

2. R — `vapply()` for type-safe functional iteration

`sapply()` is convenient but treacherous: it *guesses* the return type and can hand you a vector one day and a list the next, depending on the data. `vapply()` removes the guesswork by forcing you to declare the shape and type of each result up front. If a single iteration violates that contract, `vapply()` errors immediately instead of silently returning a malformed object that breaks code three steps downstream. In production R, that fail-fast behavior is exactly what you want.

The `FUN.VALUE` argument is the template: a prototype of one call's output. `numeric(1)` means "each call returns one number"; `character(2)` means "each returns a length-2 character vector."

```
# Compute summary stats per column with a guaranteed nume
df <- data.frame(a = c(1, 2, 3, NA), b = c(10, 20, 30, 40

stats <- vapply(
  df,
  function(col) c(mean = mean(col, na.rm = TRUE), sd = sd
  FUN.VALUE = numeric(2)  # Contract: each call returns e
)
print(stats)
# A clean 2-row matrix (mean, sd) x columns -- not an unp
```

Because the contract is explicit, `vapply()` always returns the same shape, which makes it safe inside pipelines and packages. Reach for it whenever a result feeds further computation; keep `sapply()` for throwaway interactive use only.

Reference: Advanced R (Wickham) — Functionals

3. Data-centric — Data-quality dimensions and declarative validation

Before modeling, you need to *trust* the data, and "trust" is measurable along standard **quality dimensions**: completeness (are required values present?), validity (do values conform to type/format/ range?), uniqueness (no unintended duplicates?), consistency (do related fields agree?), timeliness (is the data fresh enough?), and accuracy (does it match reality?). Naming the dimension a check targets turns vague "the data looks off" into an auditable control — and maps cleanly onto governance artifacts and data contracts.

The modern practice is **declarative validation**: express expectations as data, run them as an automated gate, and fail or quarantine on breach rather than discovering the problem after a bad model ships. A minimal expectation set for a sensor table:

```
# Declarative data-quality expectations (engine-agnostic;
table: sensor_readings
expectations:
  - column: sensor_id
    not_null: true            # Completeness.
  - column: reading_value
    type: float               # Validity (type).
    min: 0.0                   # Validity (range): negati
    max: 1000.0
  - column: recorded_at
    not_null: true
    freshness_max_lag: 24h    # Timeliness.
  - unique_together: [sensor_id, recorded_at]   # Uniquen
```

Wire this into the pipeline as a pre-load gate: passing data proceeds, failing rows are routed to a quarantine table with the failed expectation recorded. The payoff is that quality becomes a versioned, reviewable contract rather than tribal knowledge — and breaches surface where they happen, not three layers downstream.

Reference: Great Expectations — Core concepts

4. AI method — The Laplace approximation for Bayesian uncertainty
The Laplace approximation is the cheapest honest way to put error bars on a trained model. The intuition: take the log-posterior over parameters, find its peak (the MAP estimate), and approximate the whole posterior by the Gaussian that matches the curvature at that peak. Concretely, the mean is the MAP estimate and the covariance is the **inverse Hessian** of the negative log-posterior evaluated there — a single second-order computation, no MCMC, no retraining. Where the loss surface is sharp (well-determined parameters) the Gaussian is narrow; where it is flat (poorly determined) it is wide, and that width propagates into predictive uncertainty.

Use it when you already have a trained model and need calibrated uncertainty fast — especially the **last-layer** variant, where you treat only the final layer's weights as Bayesian and leave the backbone fixed. That makes it tractable for deep nets and is the basis of project idea #3 above. The main caveat is that it is a *local* Gaussian fit: reliable near the mode, but it understates uncertainty for strongly non-Gaussian or multimodal posteriors.

Today's companion notebook (`notebooks/2026-06-14-laplace-approximation.ipynb`) demonstrates the method end-to-end in pure NumPy + matplotlib: it simulates a logistic-regression problem (**M&S**), fits the MAP weights via IRLS, builds the Laplace covariance, then

marginalizes over the weight posterior and quantifies held-out accuracy and expected calibration error against the plug-in baseline (**T&E**), with an uncertainty-surface plot and a reliability diagram.

Reference: Daxberger et al., "Laplace Redux — Effortless Bayesian Deep Learning," NeurIPS 2021 (arXiv:2106.14806).

Ledgers updated: publication-ledger.md (No. 4, with No. 2–3 back-filled), project-ideas/_ledger.md (+5), and training-ledger.md (+4).

Attached: brief (.pdf) + companion notebook (.ipynb) + project briefs (.docx)
Morning Brief · Ludovici · Sunday, June 14, 2026
By Stephanie Ludovici